

Reviewer Pack — Σ^b_0 -PIND Width-2 Closure Round-Count vs DPLL Leaves

Entry fc9124ecdaa8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 08:46:08 UTC

Σ^b_0 -PIND Width-2 Closure Round-Count vs DPLL Leaves

Entry ID: fc9124ecdaa8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 08:46:08 UTC

1. Conjecture

Field A (mathematical branch): Bounded arithmetic / Cook-Reckhow-Krajicek correspondence: the Σ^b_0 -PIND round count $r_2(F)$, defined as the minimum $k \in \mathbb{N} \cup \{\infty\}$ such that iterating the closure operator $\Phi(S) = S \cup \{C : |C| \leq 2 \text{ and } C \text{ is RUP-derivable from } F \cup S \text{ in one unit-propagation pass}\}$ k times yields $\perp \in \Phi^k(\square)$; r_2 captures the depth of sharply-bounded polynomial-time induction in PV restricted to width-2 reasoning (Cook-Reckhow lineage; rarely measured directly as a quantitative invariant on small unsatisfiable CNFs). **Field B** (complexity object): Lex-DPLL search-tree leaf count $L(F)$ on small unsatisfiable 3-CNF formulas under a fixed lexicographic variable order with unit propagation, equivalently tree-resolution leaf count up to constants.

Statement:

For every unsatisfiable 3-CNF F on $n \leq 14$ variables and $m \leq 6n$ clauses: (a) if $r_2(F) < \infty$, then $\log_2 L(F) \leq 2 \cdot r_2(F) + 2 \cdot \log_2(n+1)$; (b) if $r_2(F) = \infty$, then $\log_2 L(F) \geq n/4$. Equivalently, finiteness of width-2 RUP closure forces a polynomial DPLL bound, while non-closure forces an exponential one — a sharp dichotomy.

Rationale (proposer's reasoning):

Width-2 RUP closure is the propositional shadow of Cook's PV restricted to the sharply-bounded Σ^b_0 -PIND fragment, the weakest layer of the Buss hierarchy. The Cook-Reckhow-Krajicek correspondence predicts a polynomial relation between sharply-bounded PV-proof depth and tree-like resolution size; quantifying it should expose the exact slack and yield a dichotomy mirroring the classical PHP/Tseitin obstructions to weak proof systems.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4acd2f211f6f5128

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all UNSAT instances tested (≥ 80 per (n, α) pair for $n \in \{6..12\}$, $\alpha \in \{4.5, 5.0, 6.0\}$, plus PHP_3, PHP_4, Tseitin/K_4), every formula must satisfy: if $r_2(F) < \infty$ then $\log_2 L(F) \leq$

$2 \cdot r_2(F) + 2 \cdot \log_2(n+1)$; if $r_2(F) = \infty$ (cap 200) then $\log_2 L(F) \geq n/4$. Report violation_count aggregated over all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -width-2 resolution unit propagation DPLL tree size-RUP closure bounded arithmetic PV induction proof complexity-tree resolution lower bound 3-CNF lexicographic DPLL width

Top relevant hits considered: - [http://arxiv.org/abs/cs/0209032v3] Complexity Results on DPLL and Resolution - [http://arxiv.org/abs/2407.17947v2] Supercritical Size-Width Tree-Like Resolution Trade-Offs for Graph Isomorphism - [http://arxiv.org/abs/1502.02131v2] Extracting verified decision procedures: DPLL and Resolution - [http://arxiv.org/abs/1411.7087v6] Consistency proof of a fragment of PV with substitution in bounded arithmetic - [http://arxiv.org/abs/2306.08535v1] Cyclic proofs for arithmetical inductive definitions - [http://arxiv.org/abs/2301.07061v1] A Topological Proof for a Version of Artin's Induction Theorem - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
import math
import sys
from itertools import combinations

def lex_dppll(F, unit_propagate=True):
    assignment = {}
    stack = []
    for clause in F:
        if all(lit in assignment and not assignment[lit] for lit in clause):
            return None # UNSAT
        if any(lit in assignment and assignment[lit] for lit in clause):
            continue
    unit_clauses = [lit for lit in clause if lit not in assignment and -lit not in assignment]
    if unit_clauses:
        stack.append((unit_clauses, assignment.copy()))
        assignment[unit_clauses[0]] = True
```

```

while stack:
    unit_clauses, current_assignment = stack.pop()
    new_clause = [lit for lit in unit_clauses if lit not in current_assignment and -lit not in current_assignment]
    if new_clause:
        stack.append((new_clause, current_assignment.copy()))
        current_assignment[new_clause[0]] = True
    else:
        assignment.update(current_assignment)
return assignment

def r2(F):
    rounds = 0
    while True:
        new_F = []
        for clause in F:
            if any(lit not in assignment and -lit not in assignment for lit in clause):
                new_F.append([lit for lit in clause if lit in assignment or -lit in assignment])
        if len(new_F) == len(F):
            return rounds
        F = new_F
        rounds += 1
    return math.inf

def generate_formula(n, alpha):
    variables = list(range(1, n + 1))
    clauses = []
    for i in range(1, n + 1):
        for j in range(i + 1, n + 1):
            clauses.append([i, j])
    for clause in combinations(variables, int(alpha * n)):
        clauses.append(list(clause))
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 10, 12]
    alpha_values = [4.5, 5.0, 6.0]
    results = []

    for n in n_values:
        for alpha in alpha_values:
            for _ in range(80):
                F = generate_formula(n, alpha)
                if lex_dp11(F) is None:
                    continue
                r2_value = r2(F)
                L_value = len(lex_dp11(F))
                results.append((r2_value, L_value, n))

    metric_name = "log2_L_over_r2_plus_log2_n"
    instances_tested = len(results)
    conjecture_holds = True
    counterexample = ""

    for r2_value, L_value, n in results:
        if r2_value < math.inf and not (math.log2(L_value) <= 2 * r2_value + 2 * math.log2(n + 1))

```

```

        conjecture_holds = False
        counterexample = f"r2={r2_value}, L={L_value}, n={n}"
    if r2_value == math.inf and not (math.log2(L_value) >= n / 4):
        conjecture_holds = False
        counterexample = f"r2=∞, L={L_value}, n={n}"

    metric_value = sum(math.log2(L) / (2 * r + 2 * math.log2(n + 1)) if r < math.inf else math.log2(L))

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode=124) without producing any data, so neither the support nor falsification conditions could be evaluated. The critic also challenges, and no instances were verified. | next: Reduce the search space (e.g., cap n at 10 and reduce seeds per (n, α) pair) or parallelize/optimize the r_2 closure and DPLL leaf-count routines so the test completes within the time budget.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	218468
2	preregistration	claude_max	opus	0	0	6991
3	novelty	claude_max	opus	0	0	3926
4	novelty	claude_max	opus	0	0	10881
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10724
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12612
7	judge	claude_max	opus	0	0	4643

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 268245 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/fc9124ecdaa8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fc9124ecdaa8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fc9124ecdaa8.tar.gz` (if generated)